

# Monocular 3D shape sensing for an MSCR

D435 stereo-IR ground truth · MoSSNet-style ResNet18 inference · units = mm in the left-IR rectified frame

---

## The problem

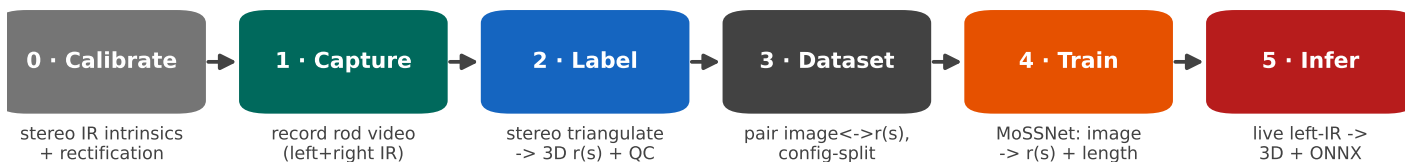
A magnetic soft continuum robot (MSCR) is a ~1 mm black rod that bends elastically under an external magnet. We want its full 3D centerline  $r(s)$  from a SINGLE camera view, fast enough for closed-loop use. A monocular image is ambiguous in depth, so a network must learn the rod's shape prior from data.

## The approach — two cameras to teach one

The Intel D435 has a calibrated STEREO INFRARED pair. We use BOTH IR views only to build accurate 3D ground-truth labels (triangulation). The trained network then infers 3D shape from the LEFT IR image ALONE — at inference the right view is never used.

*RGB camera and the IR dot projector are disabled throughout; the rod is the only thing imaged.*

## Five phases (this document follows them end to end)



Phases 0-1 are hardware setup; 2-3 manufacture training data; 4-5 are the learned model. The data quality in 2-3 dominates the final accuracy.

# 1 • Data gathering

capture.py — synchronized left/right IR frames of the rod under hand-held magnet actuation

---

## What is recorded

capture.py streams the D435 IR pair (y8, 1280×720 @30fps) with the emitter OFF, and writes raw/<session>/<left,right>/<frame>.png + a manifest. The operator sweeps a magnet to bend the rod through many shapes while it records.

## Hard-won capture constraints (the rod must be ALL of these at once)

- Distance 15–30 cm. Closer than ~13 cm → degenerate stereo (garbage depth);
- farther than ~40 cm → too little disparity. VARY distance within the band for Z range.
- Mostly VERTICAL in frame. A horizontal rod runs parallel to the epipolar scanlines, so every sample is degenerate and triangulation fails.
- Base CLAMPED & unoccluded. A hand-held base occludes the rod differently each frame, so the measured length drifts between sessions — inconsistent label noise.

## USB gotcha

At USB 2.0 the D435 cannot do 720p IR @30 ("Couldn't resolve requests"). Must be on a USB-3 port with a USB-3 cable; many USB-C cables are charge/USB-2 only.

## Sessions used for the current model (clamped base)

| session | frames | accepted | median length | note                                          |
|---------|--------|----------|---------------|-----------------------------------------------|
| s09     | 900    | 90.4 %   | 45.9 mm       | clamped, varied bend                          |
| s10     | 1030   | 100 %    | 44.6 mm       | clamped                                       |
| s11     | 917    | 100 %    | 45.0 mm       | clamped                                       |
| s03     | —      | (parked) | 55 mm         | OLD hand-held; length inconsistent → excluded |

Clamping made the rod's measured length agree across sessions (~45 mm)  
— the key that lets sessions be COMBINED without hurting the model.

## 2 · Labeling — stereo ground truth

labeler.py — turn each left/right IR pair into a metric 3D centerline  $r(s)$ , with QC

---

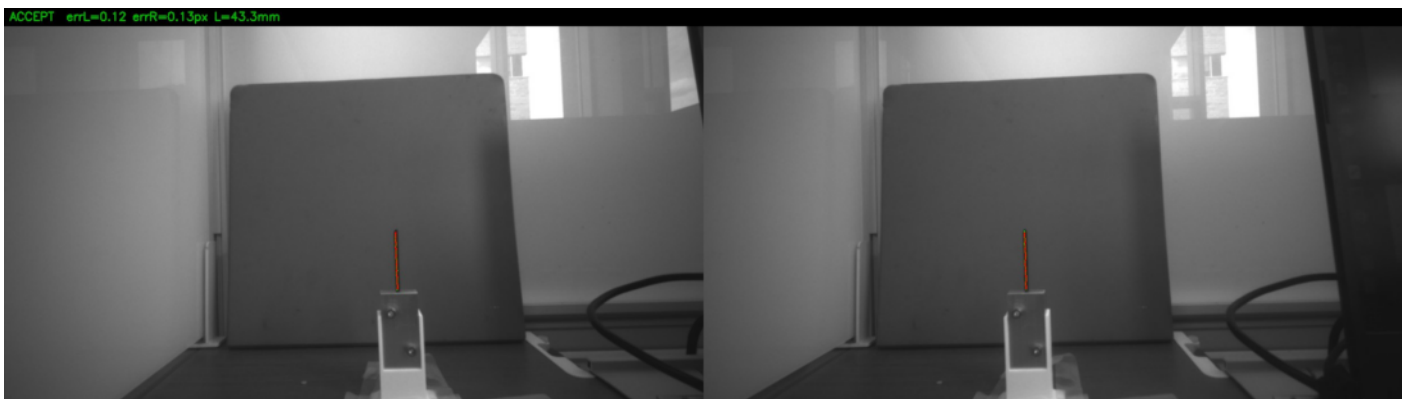
### Per-pair pipeline

- Rectify both views (calibration maps) so epipolar lines are image rows.
- Segment the rod (black-hat morphology → thin dark features, vignette-invariant).
- Pick the rod component, skeletonize, order pixels base→tip.
- Smoothing-spline + subpixel refine each 2D curve; match left↔right on scanlines.
- Triangulate correspondences ( $P1, P2$ ) → 3D points; robust depth/jump filtering.
- 3D smoothing spline, uniform-arclength resample →  $r(s)$  of  $N=64$  points.
- QC: reproject into both views; accept iff mean reproj error  $< 1.5$  px in BOTH.

### The clutter problem and its fix (base-anchor region-grow)

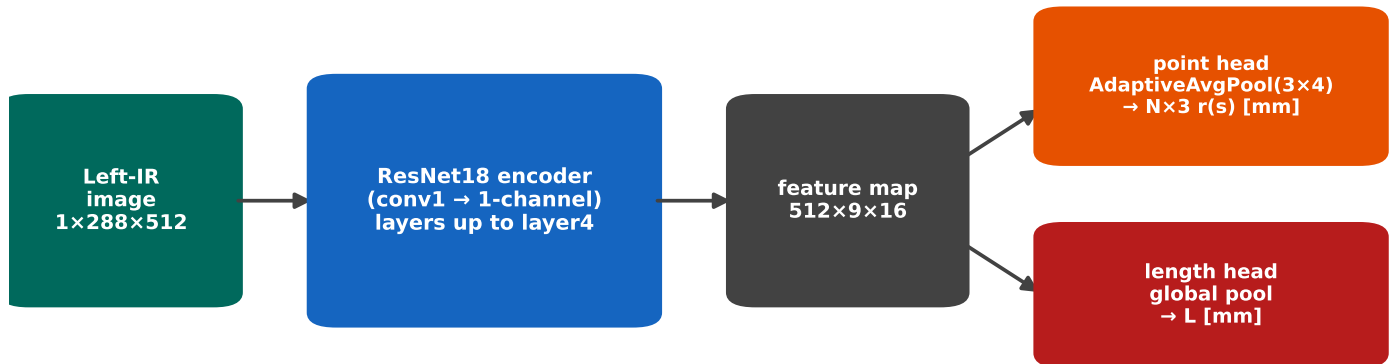
Clamping put rigid background edges in view (table lip, board edge, door molding) that out-scored the short rod on elongation — the labeler traced the wrong line. Because the clamp base is FIXED, the operator clicks it once per session (labeler.py --set-base); the labeler then keeps the connected component CONTAINING that anchor. Clutter, hand and window are separate components, so they are ignored — robust where heuristics failed.

QC overlay (left | right): green = detected 2D centerline, red = reprojected 3D  $r(s)$ , banner = accept + length



## 4a · The MoSSNet architecture

model.py — one ResNet18 encoder, two decoder heads: centerline points + scalar length



### How it maps to MoSSNet

MoSSNet (monocular soft-robot shape sensing) = a shared CNN encoder that regresses the whole centerline as an ordered point set, plus an auxiliary length output. We use a ResNet18 backbone with conv1 re-seeded to single-channel (mean of the RGB filters) so it ingests one IR channel while keeping ImageNet features.

### The spatial-head fix (why bending is now captured)

A stock ResNet ends in GLOBAL AVERAGE POOLING, which discards WHERE things are — but a rod's in-plane bend IS a position. With global pooling the net learned only depth and predicted a near-rigid rod. We drop the final avg-pool and feed the point head an AdaptiveAvgPool(3×4) grid that PRESERVES coarse position. (3×4 divides the 9×16 map evenly so ONNX export of the adaptive pool is valid.) The length head keeps a global feature since total length is position-independent.

### Output contract

- points: (B, 64, 3) — ordered base→tip centerline in mm, left-IR rectified frame.
- length: (B, 1) — total arclength in mm (auxiliary regularizer).

## 4b · Training

train.py — supervise the network with the triangulated  $r(s)$  labels

### Objective

Loss = per-axis-normalized MSE on the  $N \times 3$  points +  $\lambda \cdot$  MSE on length.

Each coordinate residual is divided by that axis' TRAIN-set std (train.normalize\_targets) so the small in-plane X/Y bending is not swamped by the large depth offset — otherwise the net nails Z in one epoch and parks at the mean in-plane shape.

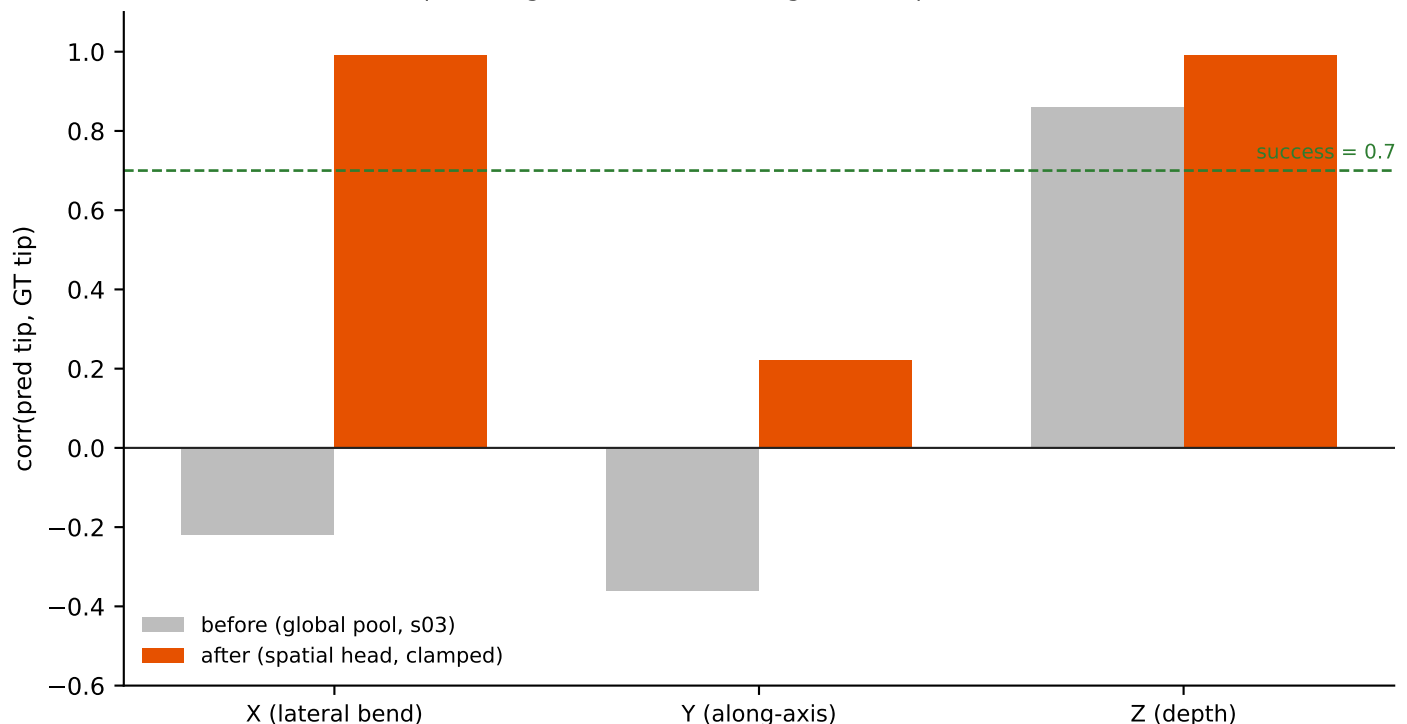
### Protocol

- Config split: whole tip-position grid CELLS go to train/val/test (not random frames),
- so near-identical poses cannot leak across splits. Photometric augmentation only
- (brightness/contrast/blur) — never geometric, because the labels are metric 3D.
- AdamW + cosine LR decay, early stop on val mean error; best-val checkpoint saved.

### Result on the clamped data (s09+s10+s11, split 1850 / 501 / 427)

TEST mean along-body error = 3.04 mm · tip error = 5.31 mm (best to date).

Per-axis tip-tracking correlation — bending is now captured (X, Z); Y still weak



## 5 · Inference & deployment

infer.py — left-IR image → 3D r(s) in real time; ONNX export; 3D visualization

### Runtime path

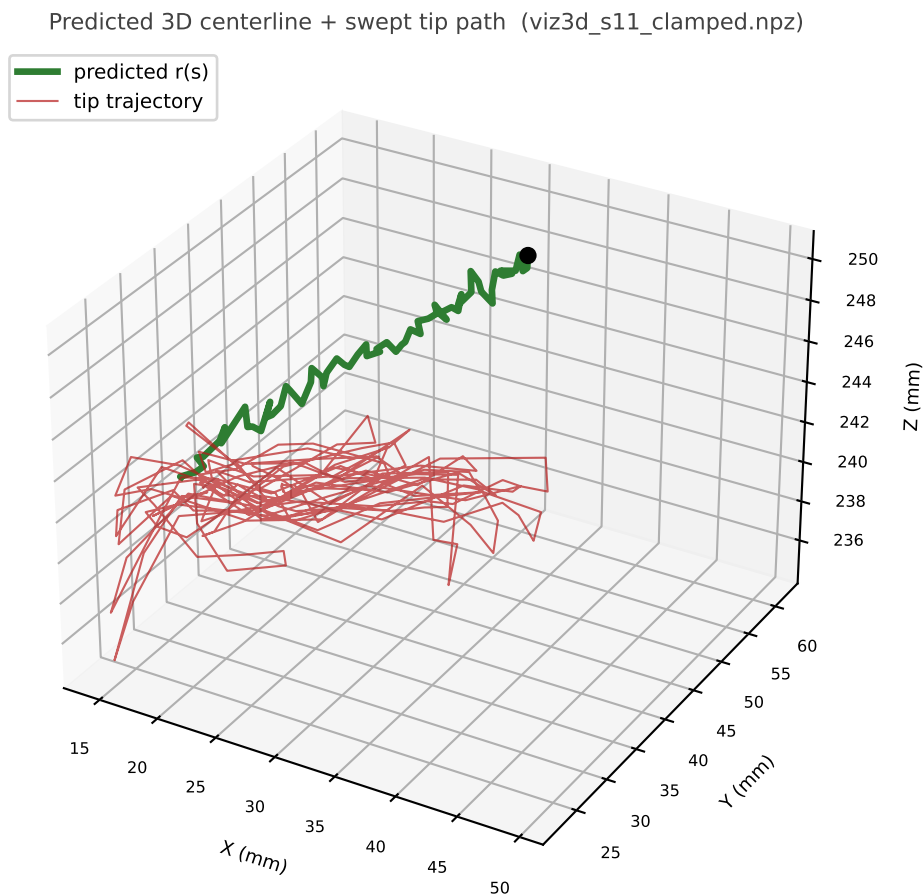
- Take ONLY the left-IR rectified image, resize to 288×512, normalize to [0,1].
- Forward pass → r(s) points in mm, length). The right view is NOT used at inference.
- Temporal EMA smoothing on the tip + a max-jump reject gate suppress per-frame jitter.
- Overlay the reprojected r(s) on the live image and report FPS.

### Portability

infer.py --export-onnx writes checkpoints/mscr\_shape.onnx and verifies torch vs onnxruntime agree to  $\sim 1e-3$ . The (3×4) adaptive pool was chosen specifically so this export is valid (a 4×4 grid fails to export on the 9×16 feature map).

### Visualization

infer.py --session <s> --viz3d renders the predicted 3D centerline (green), base (black) and tip (red) with an accumulating tip trail, and dumps r\_s\_seq [T,64,3].



# How it ties back to MoSSNet

one learned encoder, stereo-taught labels, position-aware decoding

---

## The throughline

MoSSNet's premise is that a single image of a slender body carries enough cues (apparent curvature, foreshortening, thickness) to recover its 3D shape IF the network has learned the body's deformation prior. Every phase here serves that premise:

- Stereo IR (phases 0–2) MANUFACTURES the 3D supervision a monocular net cannot get itself.
- Clamping + base-anchor labeling make that supervision CONSISTENT, so the prior is clean.
- The ResNet18 encoder is the shared MoSSNet trunk; the point head is its centerline regressor; the length head is the auxiliary geometric constraint.
- Keeping spatial features (not global pooling) is what lets the prior express BENDING, not just depth — the difference between a rigid-looking and an elastic reconstruction.

## Where it stands

X (lateral) and Z (depth) tip tracking are near-perfect (corr +0.99) with matching span.

Remaining work: the Y (camera-vertical / along-axis) tip motion is still under-tracked (corr +0.22) — likely an observability limit; needs data that bends in that plane and/or a tighter high-res ROI crop around the rod.

## Repro: the whole pipeline in one column

```
calib.py --from-device → capture.py --session SNN → labeler.py --session SNN --set-base  
→ labeler.py --session SNN → dataset.py → train.py → infer.py --session SNN [--viz3d]
```